Multisignatures as Secure as the Diffie-Hellman Problem in the Plain Public-Key Model

Duc-Phong Le[1], Alexis Bonnecaze[2], and Alban Gabillon[3]

[1] Laboratoire LIUPPA, Université de Pau et des Pays de l'Adour,
64013 Pau Cedex, France
dle@etud.univ-pau.fr

[2] Laboratoire IML, Université de Méditéranée, 13288 Marseille cedex 09 France
bonnecaze@univmed.fr

[3] Laboratoire GePaSud, Université de la Polynésie Française,
98702 FAA'A - Tahiti - Polynésie française
alban.gabillon@upf.pf

Abstract. A multisignature scheme allows a group of signers to cooperate to generate a compact signature on a common document. The length of the multisignature depends only on the security parameters of the signature schemes and not on the number of signers involved. The existing state-of-the-art multisignature schemes suffer either from impractical key setup assumptions, from loose security reductions, or from inefficient signature verification. In this paper, we present two new multisignature schemes that address all of these issues, i.e., they have efficient signature verification, they are provably secure in the plain public-key model, and their security is tightly related to the computation and decisional Diffie-Hellman problems in the random oracle model. Our construction derives from variants of EDL signatures.

1 Introduction

A multisignature scheme enables multiple signers to jointly authenticate a document producing a fixed length of digital signature. The goal of a multisignature is to prove that each member of the stated group signed the message. Multisignatures can be applied to provide efficient batch verification of several signatures of the same message under different public keys, e.g. applications concerning the multi-cast communication: IP Multi-cast, Peer-to-Peer file sharing, mobile ad hoc networks, etc.

The notion of multisignatures was first introduced by Itakura and Nakamura in [12], and has been followed by many other research works [7, 18]. Those initial schemes were not very efficient and in particular there was no formal notion of security. In fact, some effective attacks on multisignature schemes , like the *rogue key* attack, have succeeded due to some weaknesses in the key setup protocol. A rogue key attack attack can be realized whenever an adversary is allowed to choose his public key as he wishes. Typically, the adversary chooses his public key as a function of public keys of honest user, allowing him to produce forgeries easily.

H. Shacham and B. Waters (Eds.): Pairing 2009, LNCS 5671, pp. 35–51, 2009.
© Springer-Verlag Berlin Heidelberg 2009

The first formal security model for multisignatures was formalized by Micali et al. in [16]. They showed how to avoid such rogue key attacks under so-called "Knowledge of Secret Key" (KOSK) assumption, which requires the adversary to essentially provide a secret key for every public key it chooses. Their scheme implemented the KOSK assumption via an interactive pre-processing protocol involving all potential signers. This make their scheme *impractical*. Another way to realize the KOSK assumption is to employ so-called Key Registration Model (KR) for Public Key Infrastructure (PKI), introduced in the context of multisignatures by Ristenpart and Yilek [19]. In the KR model, a Certification Authority (CA) can certify a public key only if its owner passes a special key registration procedure, called a *proof of possession* of the secret key (POP). The KR model thus shifts the proof verification overhead from multisignature verifiers to the CA's. This imposes a limitation on the use of those multisignatures. Then, Bagherzandi and Jarecki [1] removed this limitation by considering an alternative mode of PKI operation which we call the Key Verification (KV) Model. In the KV model each private key owner also produces a (POP) string, but instead of handing it to the CA during the key registration process she attaches it to her key (or a PKI certificate on the key). This POP message is then verified by a multisignature receiver instead of by the CA, for example together with verification of PKI certificates on that key [1].

Plain public key model. In setting for multisignature schemes, the set of potential users should be dynamic. Users can choose their public key as they wish and may register keys at any time. In [4], Bellare and Neven discuss the drawbacks of KOSK assumption in detail and show that it is possible to dispense with this assumption. They presented a multisignature scheme which is provably secure against rogue-key attacks in the *plain public-key model*, meaning that key registration with a Certification Authority (CA) requires nothing more than that each signer has a (certified) public key. Their model allows users to register keys at any time, concurrently with other users.

Tight reduction. As Micali and Reyzin [17] put it, if the reduction is efficient and hence the relative hardness of forging and that of breaking the underlying computational assumption is close, we call the reduction *tight*. If the reduction is less efficient, we call it *close*, and if it is significantly less efficient, we call it *loose*. Intuitively, a tight reduction means that the underlying cryptographic problem is almost as hard to solve as the scheme to break.

Our contribution. In this paper, we propose two multisignature schemes which are interactive and their security is *tightly* related in the random oracle model (ROM) to, respectively, the CDH and DDH problems. In particular, our schemes are secure in the plain public-key model. In the same model, compared to the BN scheme [4], the cost of multisignature verifications of our schemes is higher than that in BN scheme. On the other hand, our schemes have tight reductions from CDH/DDH problems while the security reduction of [4] encounters a security degradation due to the use of the forking lemma. In comparison with the BNN/BGLS scheme [3, 5], our schemes have constant number

Table 1. MS scheme comparisons. For each scheme (the two last are ours) we show the assumption used to prove security, the security degradation, the protocol rounds, the type of key setup, the computational cost of verification of a multi-signature, the computational cost of signing (per signer), and the size of a multi-signature. Signature length is measured in bits, where n is the number of signers, κ is the security parameter in the BJ scheme, $|\mathbb{G}|$ is the number of bits required to represent elements in group $\mathbb{G}$, q is the group order, and $\mathbb{G}_1$ and $\mathbb{G}_2$ are two groups of points on an elliptic curve with asymmetrical bilinear maps $e : \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T$. We assume we work over a 160-bit elliptic-curve (EC) group for the DL-based schemes. For example $\kappa = 80, |\mathbb{G}| = |q| = |\mathbb{G}_1| = 160$ and $|\mathbb{G}_2| = 6 \cdot 160$. By "exp" we mean an exponentiation. (Some of the exponentiations are actually multi-exponentiations, but these have the same cost as single exponentiations.) By "pr" we mean a pairing, whose cost estimate is five 512-bit exponentiations [2].

MS Scheme	Assump	Degradation in Security	Protocol Rounds	Key Setup	Verify	Sign	Signature Length
RY+Bo [19]	GDH	$1/q_s$	1	POP	2 pr	1 exp	$\|\mathbb{G}_1\|$
RY+LOSSW[19]	GDH	$1/q_s$	1	POP	2 pr	3 exp	$\|\mathbb{G}_1\| + \|\mathbb{G}_2\|$
MOR [16]	DL	$1/q_s q_h^2$	2	POP	1 exp	1 exp	$2\|q\|$
BN [4]	DL	$1/q_h$	3	Plain	1 exp	1 exp	$\|\mathbb{G}\| + \|q\|$
BNN+BGLS [3]	GDH	$1/q_s$	1	Plain	n pr	1 exp	$\|\mathbb{G}_1\|$
BNN+BGLS [3]	GDH	tight	1	Plain	n pr	1 exp	$\|\mathbb{G}_1\| + n$ bits
BJ-CDH [1]	CDH	tight	3	POP	1 exp	1 exp	$\|\mathbb{G}\| + 2\|q\| + 2\kappa$
BJ-DDH [1]	DDH	tight	3	POP	1 exp	1 exp	$2\|q\|$
OCDH	GDH	tight	3	Plain	1 exp+2 pr	1 exp	$2\|\mathbb{G}_1\| + \|q\|$
ODDH	DDH	tight	3	Plain	2 exp	2 exp	$2\|\mathbb{G}\| + \|q\|$

of multi-exponentiations and pairings, while the multisignature verification of BNN/BGLS scheme make $O(n)$ pairing operations.

Table 1 summarizes the comparison between ours and previous multisignature schemes.

Organization. The rest of the paper is organized as follows. Section 2 provides some preliminaries about bilinear maps, Diffie-Hellman problems and the security model for multisignatures. In Section 3, we briefly recall the notion of multisignatures and their security. Section 4 presents our construction based on CDH problem and we analyze its security in Section 5. We present our multisignature scheme based on DDH problem in Section 6. Finally, we conclude the paper in Section 7.

2 Preliminary

2.1 Bilinear Map

Our first multisignature scheme uses a bilinear map, which is often called a pairing, to implement a decision procedure for the *Diffie-Hellman problem*. Typically, the pairing used is a modified Weil or Tate pairing. In this section, we briefly review the necessary facts about bilinear maps.

Let $\mathbb{G}$, $\mathbb{G}_T$ be cyclic groups of prime order p. A map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ is called an *admissible pairing* if it satisfies the following properties:

1. **bilinearity:** for all $g_1,\ g_2 \in \mathbb{G}$ and $a,\ b \in \mathbb{Z},\ e(g_1^a,\ g_2^b) = e(g_1,\ g_2)^{ab}$;
2. **non-degeneracy:** if g is a generator of $\mathbb{G}$, then $e(g, g)$ is a generator of $\mathbb{G}_T$;
3. **computable:** there exists an efficient algorithm to compute $e(g_1, g_2)$ for all $g_1, g_2 \in \mathbb{G}$.

While pairing computation is expensive, on-going algorithmic advances and hardware implementations may bring this cost down. Readers can see [6, 14] for a more detailed discussion about bilinear maps and bilinear groups.

2.2 Computational Assumptions

The security of our schemes is based on the hardness of the Diffie-Hellman problems. Let $\mathbb{G}$ be a cyclic group of prime order p and let g be a generator of $\mathbb{G}$.

Computational Diffie-Hellman. Informally, the CDH problem is to find g^{ab}, given $(g^a, g^b) \in \mathbb{G}$ as inputs, where $a, b \overset{\$}{\leftarrow} \mathbb{Z}_p^*$. An algorithm $\mathcal{A}$ has an advantage ϵ in solving the CDH problem in $\mathbb{G}$ if

$$Pr\left[\mathcal{A}(g, g^a, g^b) = g^{ab} : g \overset{R}{\leftarrow} \mathbb{G}\ ;\ a, b \overset{R}{\leftarrow} \mathbb{Z}_p^*\right]$$

is at least ϵ. We say that the CDH problem is $(t, \epsilon) - hard$ in $\mathbb{G}$ if there exists no algorithm $\mathcal{A}$ which running in time at most t have advantage ϵ in solving the CDH problem in $\mathbb{G}$.

Decisional Diffie-Hellman. The DDH problem is informally to distinguish between tuples of the form (g^a, g^b, g^{ab}) (called DDH **triples** or DDH **tuples**), where $a, b \overset{R}{\leftarrow} \mathbb{Z}_p^*$ and tuples of the form (g^a, g^b, g^c), where $a, b, c \overset{R}{\leftarrow} \mathbb{Z}_p^*$.

A distinguishing algorithm $\mathcal{A}$ has an advantage ϵ in solving the DDH problem in $\mathbb{G}$ if

$$\left|Pr\left[\mathcal{A}(g^a, g^b, g^{ab}) = 1\right] - Pr\left[\mathcal{A}(g^a, g^b, g^c) = 1\right] : a, b, c \overset{R}{\leftarrow} \mathbb{Z}_p^*\right|$$

is at least ϵ. We say that the DDH problem is $(t, \epsilon) - hard$ in $\mathbb{G}$ if there exists no distinguishing algorithm $\mathcal{A}$ which running in time at most t have advantage ϵ in solving the DDH problem in $\mathbb{G}$.

The DDH assumption is stronger than the CDH assumption, that is, if the CDH problem is efficiently solved in $\mathbb{G}$ then the DDH problem is also solved efficiently in $\mathbb{G}$. The inverse of these statements is not believed to be true in general. Indeed, Joux and Nguyen [14] showed that there are groups (called gap Diffie-Hellman (GDH) groups) for which the DDH is *e*asy by using an efficiently computable bilinear map e, yet the CDH in the group is still believed to be hard.

3 Multisignature Scheme and Its Security Model

3.1 Multisignature Scheme

Formally, a multisignature scheme consists of four algorithms $\mathsf{MS} = \mathsf{Setup}, \mathsf{KGen}, \mathsf{MSign}, \mathsf{Vrfy}$.

- $\mathsf{params} \rightarrow \mathsf{Setup}(1^k)$. A central authority, on input the security parameter k, runs the algorithm Setup to produces the global information params. Algorithm Setup is probabilistic.
- $(sk, pk) \leftarrow \mathsf{KGen}$, executed by each signer on input params, generates this signer's secret key sk, the corresponding public key pk. Algorithm KGen is probabilistic.
- The *multi-signing algorithm* MSign might be a probabilistic algorithm which, given a message m, the global information params and a list of signers L along with their public and secret keys, produces a multisignature σ. The multi-signing can be interactive or non-interactive.
- $\{0, 1\} \leftarrow \mathsf{Vrfy}(\mathsf{params}, m, L, \sigma)$ verifies whether σ is a valid multisignature on the message m with respect to L. This algorithm is deterministic.

3.2 Multisignature Security in Plain Public-Key Model

The attacks of an adversary $\mathcal{A}$ against multisignature schemes are to forge a group of signers L and a multisignature of some message such that the latter is accepted by a verifier whereas some signers of the group L did not sign the message. We give the adversary the power to request the private key on all but one signer and its goal is to frame this honest signer. The adversary can choose their public keys arbitrarily, even as a function of the public key of the honest signer. The adversary $\mathcal{A}$ is given the global information params, a challenging public key pk^* corresponding to the honest signer and signing and hash oracles. His goal is to output a forged message-group-multisignature tuple (m, L, σ), such that the honest signer, who did not complete the multisignature generation protocol on the input message m, is in L and $\mathsf{MS.Vrfy}(\mathsf{params}, m, L, \sigma) = 1$.

Let $\mathcal{A}$ be an adversary against the multisignature scheme, which consists of four algorithms Setup, KGen, MSign, and Vrfy. As in the previous works on multisignatures, e.g. [1, 16], we define multisignature security as Universal Unforgeability (UU) under a Chosen Message Attack (CMA) against a single honest player. Namely, we define $Adv_{\mathsf{MS}}^{\mathsf{uu-cma}}(\mathcal{A})$ to be the probability that experiment $\mathbf{Exp}_{\mathsf{MS}}^{\mathsf{uu-cma}}(\mathcal{A})$ described in Table 2 outputs 1. A multisignature scheme is said to be $(t, q_S, q_H, N, \epsilon)$-secure in the random oracle model if $Adv_{\mathsf{MS}}^{\mathsf{uu-cma}}(\mathcal{A}) \leq \epsilon$ for every adversary $\mathcal{A}$ that runs in time at most t, makes at most q_S signing queries with the honest signer, at most q_H random oracle queries, and the number of signers in L involved in any signing query or in the forgery is at most N.

Table 2. Chosen Message Attack against Multisignature in the Plain public-key model

Experiment $\mathbf{Exp}_{\mathsf{MS}}^{\mathsf{uu-cma}}(\mathcal{A})$:

$\mathsf{params} \leftarrow \mathsf{Setup}(1^k)$; $(sk^*, pk^*) \leftarrow \mathsf{KGen}(\mathsf{params})$; $List \leftarrow \emptyset$;
Run $\mathcal{A}(\mathsf{params}, pk^*)$, and for every signature query m made by $\mathcal{A}$ do the following:
1. $List \leftarrow List \cup \{(m, L)\}$, where L is the list of users participating in signing the message m;
2. Execute protocol MSign on behalf of an honest player on inputs $(\mathsf{params}, m, sk^*, L)$, forwarding messages to and from $\mathcal{A}$.
When $\mathcal{A}$ halt; parse its outputs as (m, L, σ).
If $(m, L) \notin List$, $pk_1 = pk^*$ and $\mathsf{Vrfy}(\mathsf{params}, m, L, \sigma) = 1$ then return 1. Otherwise return 0.

4 A Multisignature Scheme Based on the CDH Problem

4.1 The Chevallier-Mames Signature Scheme

In order to give some intuition into our scheme, we briefly recall the variant of *EDL* signature scheme presented in [9].

Let $\mathbb{G}$ be a cyclic group of prime order p, g be a generator of $\mathbb{G}$ and let $\mathcal{H}, \mathcal{G}$ be two collision-resistant hash functions. To sign a message m, a signer U, having private/public key pair (x, y), does as follows:

- chooses $k \in \mathbb{Z}_p$ at random;
- computes $u = g^k$, $h = \mathcal{H}(u)$, $z = h^x$ and $v = h^k$;
- queries $c = \mathcal{G}(m, g, h, y, z, u, v)$ and computes $s = k + cx$;
- outputs $\sigma = (z, s, c) \in \mathbb{G} \times \mathbb{Z}_p^2$ as the signature of m.

To verify a signature $\sigma = (z, s, c)$ for m, one computes $u' = g^s y^{-c}$, $h' = \mathcal{H}(u')$ and $v' = h'^s z^{-c}$. The signature σ is accepted iff $c = \mathcal{G}(m, g, h', y, z, u', v')$.

The Chevallier-Mames signature scheme [9] is the most efficient in variants of *EDL* scheme [8, 10, 11, 13, 15] under CDH assumption.

4.2 Our Multisignature Scheme

In our multisignature generation protocol, each signer computes and uses an independent *challenge* $c_i = \mathcal{G}(y_i, L, u, m, g, h)$ in the proofs of knowledge of equality of discrete logarithms. This way pointed out by Bellare and Neven [4], allows us to avoid KOSK and KR models. Thus, we first modify the Chevallier-Mames signatures as follows: let a signature of a message m under public key $y \in \mathbb{G}$ be a quadruplet $(u, v, z, s) \in \mathbb{G}^3 \times \mathbb{Z}_p$ such that $g^s = uy^c$ and $h^s = vz^c$, where $h = \mathcal{H}(u)$ and $c = \mathcal{G}(m, g, h, y, z, u, v)$. In order to aggregate individual signatures of a common message m, (u_i, v_i, z_i, s_i), for $1 \leq i \leq n$ under public keys $PK = \{y_1, y_2, \cdots, y_n\}$, we may let a multisignature be a tuple $(u, v, s, \{z_i\}_{i=1}^n)$ such that:

$g^s = u \cdot \prod_{i=1}^n y_i^{c_i}$ and $h^s = v \cdot \prod_{i=1}^n z_i^{c_i}$, where $u = \prod_{i=1}^n u_i, v = \prod_{i=1}^n v_i$ and $s = \sum_{i=1}^n s_i$. Because challenges c_i are different, we cannot aggregate individual shares z_i. The size of the multisignature thereby grows linearly with the number of signers. To solve this problem, we propose to use a pairing. A multisignature may be a triple $(u, z, s) \in \mathbb{G}^2 \times \mathbb{Z}_p$ such that: $g^s = u \cdot \prod_{i=1}^n y_i^{c_i}$ and $e(z, g) = e(h, \prod_{i=1}^n y_i)$, where $h = \mathcal{H}(u)$, $c_i = \mathcal{G}(y_i, L, u, m, g, h)$. The values of u, z (and s) are typically computed as the product (the sum resp.) of individual shares of u_i, z_i (of s_i resp.) contributed by each signer.

In describing the scheme, we assume the signers directly send and receive messages to each other over a point-to-point network. Like in [4], to avoid using the rewinding technique in security proof, our scheme requires an additional communication round between signers, in which each signer first makes an additional random oracle query on its individual share u and then sends this *challenge* to every other signer before sending u. This prevents the forger to know the value of individual share u before the simulator does. The simulator thereby could imitate the oracle so as to produce commitments and challenges simultaneously.

Let $\mathbb{G}, \mathbb{G}_T$ be cyclic groups of prime order p in which $\mathbb{G}$ provides admissible parings, let k be a security parameter. Three cryptographic hash functions: $\mathcal{H}_0 : \mathbb{G} \to \{0,1\}^{l_0}, \mathcal{H}_1 : \mathbb{G} \to \mathbb{G}$ and $\mathcal{G} : \{0,1\}^* \to \mathbb{Z}_p$. We remark that $\mathcal{H}_0$, $\mathcal{H}_1$ and $\mathcal{G}$ will be viewed as random oracles in our security proof. The multisignature scheme $\mathsf{MS} = \mathsf{Setup}, \mathsf{KGen}, \mathsf{MSign}, \mathsf{Vrfy}$ works as follows:

Parameter generation (Setup)**:** A trusted center generates a random generator $g \in \mathbb{G}^*$ and publishes $\mathsf{params} = (\mathbb{G}, \mathbb{G}_T, e, g, \mathcal{H}_0, \mathcal{H}_1, \mathcal{G})$ as system wide parameters.

Key generation (KGen)**:** On input 1^k, each signer picks a random number $x \xleftarrow{R} \mathbb{Z}_p$ as his private key. The corresponding public key is $y = g^x$.

Signing (MSign)**:** Suppose that $L = \{P_1, P_2, \ldots, P_n\}$ is a group of n signers that wish to sign a common message m, each having as input its own public and secret key as well as a multiset of public keys $Pk = \{y_1, y_2, \ldots, y_n\}$ of the other signers. We also stress that the signers $P_1, \ldots, P_n$ are merely local references to co-signers, defined by one signer within one protocol instance. The signing process, which is *interactive*, consists of three rounds:

Round 1. Each signer $P_i \in L$:
- picks a random number $r_i \in Z_p$;
- computes its individual commitment $u_i = g^{r_i}$;
- queries $\mathcal{H}_0$ to compute the challenge $h_i = \mathcal{H}_0(u_i)$;
- sends h_i to every other signer.

Round 2. Each signer $P_i \in L$:
- receives h_j from signer j, for $1 \le j \le n, j \ne i$;
- sends u_i to signer j.

Round 3. Each signer $P_i \in L$:
- receives u_j from signer j, for $1 \le j \le n, j \ne i$;
- checks whether $h_j = \mathcal{H}_0(u_j)$ for all $1 \le j \le n, j \ne i$. If not, abort the protocol. Otherwise,
- computes $u = \prod_{i=1}^n u_i$, $h = \mathcal{H}_1(u)$ and $z_i = h^{x_i}$.

- queries $c_i = \mathcal{G}(y_i, u, Pk, m, g, h)$ and computes $s_i = r_i + x_i c_i \bmod p$.
- sends to signer j: $z_i,\ s_i$, for $1 \le j \le n, j \ne i$.

After receiving $z_j,\ s_j$ from signer j, for $1 \le j \le n, j \ne i$, each signer $P_i \in L$:
- computes $z = \prod_{i=1}^{n} z_i$, $s = \sum_{i=1}^{n} s_i \bmod p$;
- outputs the signature $\sigma = (u, z, s)$;

Verification (Vrfy)**:** To verify a signature σ of a message m of a group L, whose public keys is the multiset $Pk = \{y_1, \ldots, y_n\}$, one does as follows:
- Compute $h = \mathcal{H}_1(u)$;
- Compute $c_i = \mathcal{G}(y_i, u, Pk, m, g, h)$ for all $1 \le i \le n$;
- Check whether:

$$g^s = u \cdot \prod_{i=1}^{n} y_i^{c_i} \quad \text{and} \quad e(z, g) = e(h, \prod_{i=1}^{n} y_i).$$

5 Security Analysis

In this section, we reduce the security of the proposed multisignature scheme to the CDH problem in the group $\mathbb{G}$ with bilinear map e. The main technique used to obtain a tight proof of security is to prove equality of discrete logarithms (see [11] for a discussion more details). Let N be the maximum number of signers which participate in signing in one protocol instance, the following theorem implies that the proposed multisignature scheme is secure if the CDH assumption is held in $\mathbb{G}$.

Theorem 1. *The proposed multisignature scheme is $(t, q_H, q_S, N, \epsilon)$-unforgeable if the* CDH *problem is (t', ϵ')-unforgeable in $\mathbb{G}$, where*

$$\epsilon' \ge \epsilon - \frac{(q_H + Nq_S + 1)^2}{2^{l_0}} - \frac{q_S((N+1)q_H + 2q_S)}{q},$$

and

$$t' \le t + 6q_S t_{exp} + O((q_S + q_H)(1 + q_H + Nq_S)),$$

where t_{exp} is the time of an exponentiation in $\mathbb{G}$.

Proof. We are given a group $\mathbb{G}$ and a CDH challenge (g, g^x, g^a). Let $\mathcal{A}$ be a polynomial time forger that (t, q_H, q_S, ϵ)-breaks the proposed scheme. We need to construct an algorithm $\mathcal{B}$ which, by interacting with the adversary $\mathcal{A}$, (t', ϵ')-breaks this challenge, i.e. to find g^{ax}. The forger $\mathcal{A}$, after q_H hash queries to random oracles ($\mathcal{H}_0$, $\mathcal{H}_1$ and $\mathcal{G}$) and q_S signature queries, is able to produce a multisignature forgery with probability ϵ within time t.

Assume that $\mathcal{A}$ is trying to attack the honest signer P^*. $\mathcal{B}$ runs the forger $\mathcal{A}$ on input system parameters and target public key $y^* = g^x$. Like [4], we make use of a list T which assigns a unique index $1 \le i \le q_H + Nq_S$ to each public key y occurring either as a cosigner's public key in one of $\mathcal{A}$'s signature queries, or as the first item in the argument of one of $\mathcal{A}$'s queries to $\mathcal{G}$ and a table $\mathsf{G}[\cdot\,,\cdot]$ which is used to simulate the random oracle $\mathcal{G}$. Algorithm $\mathcal{B}$ uses a counter *ctr* indicating the current index of this list, initially set to zero. $\mathcal{B}$ assigns $\mathsf{T}[y^*] \leftarrow 0$. It responds to $\mathcal{A}$'s oracle queries, essentially, at random as follows:

Queries to $\mathcal{H}_0$. In response to a query $\mathcal{H}_0(u_i)$, $\mathcal{B}$ first checks if the output of $\mathcal{H}_0$ on this input has been previously defined. If so, $\mathcal{B}$ returns the previously assigned value. Otherwise, $\mathcal{B}$ returns with a value chosen uniformly at random from $\{0,1\}^{l_0}$. All queries u_i are stored in a called list H.

Queries to $\mathcal{H}_1$. In response to a query from the forger $\mathcal{A}$ to $\mathcal{H}_1(u)$, algorithm $\mathcal{B}$ generates a random number $d \in Z_p$, and returns $(g^a)g^d$. All queries u are stored in a list called U.

Queries to $\mathcal{G}$. In response to a query $\mathcal{G}$, we first parse the argument of the query into two portions as y and Q. If $\mathsf{T}[y]$ is undefined then $\mathcal{B}$ increases ctr and sets $\mathsf{T}[y] \leftarrow ctr$. If $\mathsf{G}[ctr, Q]$ has not yet been defined, then $\mathcal{B}$ assigns $\mathsf{G}[i, Q]$, for all $1 \leq i \leq q_H + Nq_S$ with random numbers, and picks in advance at random $e_1, \ldots, e_{q_H+q_S} \in \mathbb{Z}_p$ to assign for $\mathsf{G}[0, Q]$.

Signing query on m with group of users L: Signature queries to the honest signer P^* consists of three rounds. First, the adversary provides m, L to P^* and receives the individual challenge h^* from P^* in response. Second, playing the role of rest signer, the adversary $\mathcal{A}$ provides the challenges h_i to P^* and receives u^* from P^* in response. Third, the adversary provides the commitments u_i to P^* and receives z^*, s^* from P^* in response. Note that in the simulation, rewinding is not required since the joint commitment u is not provided to the simulator by the adversary. In detail, answering signature queries works as follows:

First, $\mathcal{B}$ checks whether $P^* \notin L$, if so algorithm $\mathcal{B}$ returns $\perp$ to $\mathcal{A}$. If not, it parses the public keys of signers in L as $Pk = \{y_1 = y^*, y_2, \ldots, y_n\}$. Then, $\mathcal{B}$ checks whether $\mathsf{T}[y_i]$, for $i \in \{2, \ldots, n\}$, has already been defined. If not, it increases ctr and sets $\mathsf{T}[y_i] \leftarrow ctr$. Then, $\mathcal{B}$ sets c_1 at random as $e_1, \ldots, e_{q_H+q_S}$ in advance. $\mathcal{B}$ generates $(\gamma, s_1) \in \mathbb{Z}_p^2$ at random, computes $u_1 = g^{s_1} y^{-c_1}$. It sets $h_1 = \mathcal{H}_0(u_1)$ and sends it to all signers.

After receiving $h_2, \cdots, h_n$ from the adversary $\mathcal{A}$, $\mathcal{B}$ looks up in the list H for values u_j such that $h_i = \mathcal{H}_0(u_j)$. If multiple such values are found for some i, the algorithm $\mathcal{B}$ stops (**Event 1**). If no such value was found for some i then it sets $alert \leftarrow true$ and sends u_1 to all co-signers; otherwise, $\mathcal{B}$ computes $u = \prod_{i=1}^{n} u_i$. If $\mathcal{H}_1(u)$ is already set, algorithm $\mathcal{B}$ fails and stops (**Event 2**). Else, algorithm $\mathcal{B}$ sets $h = \mathcal{H}_1(u) = g^\gamma$ and computes $z_1 = y_1^\gamma = (g^x)^\gamma = h^x$, remark that $DL_g(y) = DL_h(z)(= x)$. Then, $\mathcal{B}$ checks whether $\mathsf{G}[0, Q]$ has already been defined for $Q = \langle u, Pk, m, g, h\rangle$. If so, it fails and stops (**Event 3**). If not, it sets $\mathcal{G}(y_1, u, Pk, m, g, h) = \mathsf{G}[0, Q] = c_1$, randomly chooses $\mathsf{G}[i, Q] \xleftarrow{R} \mathbb{Z}_p$ for all $1 \leq i \leq q_H + Nq_S$ and sends u_1 to all co-signers.

After receiving $u_2, \ldots, u_n$ from $\mathcal{A}$, $\mathcal{B}$ verifies that $h_i = \mathcal{H}_0(u_i)$ for all $1 \leq i \leq n$. If not, it returns $\perp$ to $\mathcal{A}$. If $alert = true$, $\mathcal{B}$ fails and stops (**Event 4**). Else, it sends (z_1, s_1) to all co-signers.

After receiving $(z_2, s_2), \cdots, (z_n, s_n)$ from co-signers ($\mathcal{A}$), $\mathcal{B}$ computes $z = \prod_{i=1}^{n} z_i$ and $s = \sum_{i=1}^{n} s_i$ and returns the valid signature (u, z, s). All u's computed during signature queries are stored in a list called Y.

As we can see, this simulator is valid, except for some events:

- **Event 1:** In this case, there exist two values $u_i \neq u'_i$ such that $h_i = \mathcal{H}_0(u_i) = \mathcal{H}_0(u'_i)$ for some i, i.e, there is at least one collision occurred in $\mathcal{H}_0$. As outputs of $\mathcal{H}_0$ are chosen at random from $\{0,1\}^{l_0}$ and since there are at most $q_{\mathcal{H}_0} + Nq_S$ queries to $\mathcal{H}_0$, the probability that at least one collision occurs is upper bounded by $\frac{(q_{\mathcal{H}_0}+Nq_S)(q_{\mathcal{H}_0}+Nq_S+1)/2}{2^{l_0}} \leq \frac{(q_{\mathcal{H}_0}+Nq_S+1)^2}{2^{l_0+1}}$.
- **Event 2:** As u is a random element in $\mathbb{G}$, the probability that the $\mathcal{H}_1(u)$ is already set is less than $\frac{q_{\mathcal{H}_1}+q_S}{p}$, for one signature query. For q_S signature queries, the failure probability is thus upper bounded by $\frac{q_S(q_{\mathcal{H}_1}+q_S)}{p}$.
- **Event 3:** Algorithm $\mathcal{B}$ only aborts at event 3 if it has run into an input string $\langle 0, u, Pk, m, g, h\rangle = \langle 0, u, Pk, m, g, g^\gamma\rangle$ on which $\mathcal{G}$ has been already queried, for $\gamma \in \mathbb{Z}_q^*$. But as Event 2 did not happened, $\mathcal{H}_1(u)$ have not yet been defined, and so γ is absolutely unknown for the adversary. Then, the probability that $\mathcal{G}(0, u, Pk, m, g, h)$ is already set is less than $\frac{q_{\mathcal{G}}+q_S}{p}$, for one signature query. For q_S signature queries, the failure probability is thus upper bounded by $\frac{q_S(q_{\mathcal{G}}+q_S)}{p}$.
- **Event 4:** In this case, $\mathcal{A}$ must have predicted the value of $\mathcal{H}_0(u_i)$ for at least one $1 \leq i \leq n$, which it can do with probability at most $N/2^{l_0}$, for one signature query. For q_S signature queries, the failure probability is thus upper bounded by $q_S N/2^{l_0}$.

As a conclusion, except with a failure probability:

$$\begin{aligned}\delta_{stop} &= \frac{(q_{\mathcal{H}_0} + Nq_S + 1)^2}{2^{l_0+1}} + \frac{q_S(q_{\mathcal{H}_1} + q_S)}{p} + \frac{q_S(q_{\mathcal{G}} + q_S)}{p} + \frac{q_S N}{2^{l_0}} \\ &\leq \frac{(q_{\mathcal{H}_0} + Nq_S + 1)^2}{2^{l_0}} + \frac{q_S(q_{\mathcal{H}_1} + q_{\mathcal{G}} + 2q_S)}{p},\end{aligned}$$

the simulation is successful.

Eventually, $\mathcal{A}$ halts and outputs an attempted forgery $\sigma = (\hat{u}, \hat{z}, \hat{s})$ on some message $\hat{m}$ along with $L = \{P^*, P_2, \cdots, P_n\}$. It must not previously have requested a signature on $\hat{m}$ with L. In addition, it outputs the private keys $(x_2, \cdots, x_n)$ for all secret keys except the key x of the challenge P^*. Algorithm $\mathcal{B}$ first computes additional random oracle queries $\mathcal{G}_1(y_i, \hat{u}, Pk, \hat{m}, g, \hat{h})$ for $1 \leq i \leq n$, thereby making sure that $\mathsf{T}[y_i]$ is defined.

We divide into two cases : either $\hat{u}$ belongs to the list U or $\hat{u}$ belongs to the list Y.

In the first case, the algorithm $\mathcal{B}$ first computes $\hat{h} = \mathcal{H}_1(\hat{u})$, and then $\hat{z}_1 = \hat{z}/\prod_{i=2}^n \hat{h}^{x_i}$. If $\mathcal{A}$'s forgery is valid, the algorithm $\mathcal{B}$ returns $(\hat{u}, \hat{z}, \hat{s}, \hat{h}, \hat{z}_1)$. We argue that, with all but negligible probability, $\hat{z}_1 = \hat{h}^x$; if so, say $\hat{z}_1$ is *good*. Indeed, if $\hat{z}_1$ is not good then for any A, B there is at most one possible value of c for which there exists an s satisfying $A = g^s y^c$ and $B = \hat{h}^s \hat{z}_1^c$ (*lemma* 1 in [11]). If $\hat{z}_1$ is not good, then, for any hash query $\mathcal{G}(y_1, \hat{u}, Pk, \hat{m}, g, \hat{h})$ made by $\mathcal{B}$ the probability that the query returns a c for which there exists an s as above is

at most $1/p$. It follows that the probability that $\mathcal{B}$ outputs a valid forgery where $\hat{z}_1$ is not good is at most $q_{\mathcal{G}}/p$. Otherwise, the problem CDH is solved as follows:

$$\frac{\hat{z}_1}{y_1^d} = \frac{\hat{h}^{x_1}}{y_1^d} = \frac{(g^a g^d)^x}{(g^x)^d} = g^{ax},$$

As a conclusion, in the first case, except with a failure probability $\delta_1 = \frac{q_{\mathcal{G}}}{p}$, the forgery will be used to successfully solve the CDH problem.

In the second case, $\hat{u}$ is a member of Y. This case can happen, as there is no message in the input of $\mathcal{H}_1$, and so we can imagine that the attacker reuses a u that corresponds to a u of a signature given by the signature oracle. Then, the algorithm $\mathcal{B}$ can recover from its log files all quantities that correspond to this $u = \hat{u}$, i.e., $(h, \{(z_i, s_i, c_i)\}_{i=1..n}, m)$.

At this moment, we can see that we have $u = g^s \prod_{i=1}^n y_i^{-c_i} = \hat{u} = g^{\hat{s}} \prod_{i=1}^n y_i^{-\hat{c}_i}$. It is exactly the kind of hypothesis that is used by the forking lemma to prove a (loose) security. But here, this equality is not obtained by restarting the attacker (as it is done in the forking lemma), but just by construction. More precisely, we can recover easily the private key x, as far as $\hat{c}_i \neq c_i \bmod p$.

As the message-list of signer pair $(\hat{Pk}, \hat{m})$ is new, $c_i \neq \hat{c}_i$ for $1 \leq i \leq n$ or a collision on $\mathcal{G}$ function happened, between a $\mathcal{G}$ returned the signature simulation and a $\mathcal{G}$ returned by a direct $\mathcal{G}$ query, which occurs with a probability smaller than $\frac{Nq_S \cdot q_{\mathcal{G}}}{p}$. Hence, except an error with a probability smaller than $\delta_2 = \frac{Nq_S \cdot q_{\mathcal{G}}}{p}$, we have $\hat{c}_i \neq c_i$, and so we can recover the private key x. Equation $s - \sum_{i=1}^n x_i c_i = \hat{s} - \sum_{i=1}^n x_i \hat{c}_i \pmod p$ gives $x = x_1 = \frac{s-\hat{s}}{c_1 - \hat{c}_1} - \sum_{i=2}^n x_i \frac{c_i - \hat{c}_i}{c_1 - \hat{c}_1} \bmod p$. We can see that this second case gives not only the solution to the CDH challenge, but also the solution to the discrete logarithm.

Summing the probabilities, we can see that in both cases, the algorithm $\mathcal{B}$ can use the forgery given by the adversary to solve the CDH. The success probability ϵ' satisfies $\epsilon' \geq \epsilon - \delta_{stop} - \max(\delta_1, \delta_2)$. Thus,

$$\begin{aligned}
\epsilon' &\geq \epsilon - \frac{(q_{\mathcal{H}_0} + Nq_S + 1)^2}{2^{l_0}} - \frac{q_S(q_{\mathcal{H}_1} + q_{\mathcal{G}} + 2q_S)}{q} - \max(\frac{q_{\mathcal{G}}}{q}, \frac{Nq_S \cdot q_{\mathcal{G}}}{q}) \\
&\geq \epsilon - \frac{(q_{\mathcal{H}_0} + Nq_S + 1)^2}{2^{l_0}} - \frac{q_S(q_{\mathcal{H}_1} + (N+1)q_{\mathcal{G}} + 2q_S)}{q} \\
&\geq \epsilon - \frac{(q_H + Nq_S + 1)^2}{2^{l_0}} - \frac{q_S((N+1)q_H + 2q_S)}{q},
\end{aligned}$$

and the running time t' satisfies

$$t' \leq t + 6q_S t_{exp} + O((q_S + q_H)(1 + q_H + Nq_S)),$$

where $q_H = q_{\mathcal{H}_0} + q_{\mathcal{H}_1} + q_{\mathcal{G}}$, t_{exp} is the time of an exponentiation in $\mathbb{G}$.

6 A Multisignature Scheme Based on the DDH Problem

In the previous scheme, our scheme makes use of GDH groups. In this section, we present a more efficient multisignature scheme which relies on decisional

Diffie-Hellman problem, stronger than CDH assumption, in non-pairing groups. Our construction is based on Katz-Wang signature scheme [15] that works as follows:

Let $\mathbb{G}$ be a cyclic group of prime order p, g be a generator of $\mathbb{G}$, $h \in \mathbb{G}$ chosen randomly and let $\mathcal{H} : \{0,1\}^* \rightarrow \{0,1\}^{l_0}$ be a hash function. A Katz-Wang signature of a message m under public keys (y_1, y_2) is a triplet (A, B, s), such that $g^s = Ay_1^c$ and $h^s = By_2^c$, where $A = g^r$, $B = h^r$ and $c = \mathcal{H}(A, B, m)$.

We slightly modify the Katz-Wang signature [15] scheme for easily extending it to multisignatures. The idea of using the Katz-Wang signatures for constructing multisignatures was first suggested by Bellare and Neven in section 6 of [4] as further results.

6.1 Our Multisignature Scheme

As before, we assume that $\mathbb{G}, \mathbb{G}_T$ be cyclic groups of prime order p, k be a security parameter. Two cryptographic hash functions: $\mathcal{H} : \mathbb{G} \rightarrow \{0,1\}^{l_0}$ and $\mathcal{G} : \{0,1\}^* \rightarrow \mathbb{Z}_p$. Our second scheme is defined as follows:

Parameter generation. A trusted center chooses a generator $g \in \mathbb{G}^*$ and $h \in \mathbb{G}$ at random. It then publishes $params = (\mathbb{G}, e, g, h, \mathcal{H}, \mathcal{G})$ as system wide parameters.

Key generation. On input 1^k, each signer picks a random number $x_i \stackrel{R}{\leftarrow} \mathbb{Z}_p$ as his private key. The corresponding public keys are $PK_i = (y_i, z_i)(= (g^{x_i}, h^{x_i}))$.

Signing. Suppose that $L = \{P_1, P_2, \ldots, P_n\}$ is a group of n signers that wish to sign a common message m, each having as input its own public and secret key as well as a multiset of public keys $Pk = \{PK_1, \ldots, PK_n\}$ of the other signers. We also stress that the signers $P_1, \ldots, P_n$ are merely local references to co-signers, defined by one signer within one protocol instance. The signing process, which is *interactive*, consists of four rounds, where in each round signers send (and receive) a message to (from resp.) each other signer.

Round 1. Each signer $P_i \in L$:
- picks a random number $r_i \in Z_p$;
- computes its individual commitments $u_i = g^{r_i}$ and $v_i = h^{r_i}$, then queries $\mathcal{H}$ to compute challenges $h_i = \mathcal{H}(u_i)$ and $t_i = \mathcal{H}(v_i)$;
- sends h_i, t_i to every other signer.

Round 2. Each signer $P_i \in L$:
- receives h_j, t_j from signer j, for $1 \leq j \leq n, j \neq i$;
- sends u_i, v_i to signer j.

Round 3. Each signer $P_i \in L$:
- receives u_j, v_j from signer j, for $1 \leq j \leq n, j \neq i$;
- checks whether $h_j = \mathcal{H}(u_j)$ and $t_j = \mathcal{H}(v_j)$ for all $1 \leq j \leq n, j \neq i$. If not, abort the protocol. Otherwise, computes $u = \prod_{i=1}^n u_i$ and $v = \prod_{i=1}^n v_i$.
- queries $c_i = \mathcal{G}(PK_i, u, v, Pk, m, g, h)$ and computes $s_i = r_i + x_i c_i \mod p$.
- sends to signer j: s_i.

After receiving s_j from signer j, each signer $P_i \in L$:
- computes $s = \sum_{i=1}^{n} s_i \bmod p$;
- outputs the signature $\sigma = (u, v, s)$;

Verification. Given the valid signature σ, list of group of users L and message m, the verifier computes $c_i = \mathcal{G}(PK_i, u, v, Pk, m, g, h)$ for all $1 \leq i \leq n$ and tests whether: $g^s = u \cdot \prod_{i=1}^{n} y_i^{c_i}$ and $h^s = v \cdot \prod_{i=1}^{n} z_i^{c_i}$.

6.2 Security

Theorem 2. *The proposed multisignature scheme is* $(t, q_H, q_S, N, \epsilon)$*-unforgeable if the* DDH *problem is* (t', ϵ')*-unforgeable in* $\mathbb{G}$*, where*

$$\epsilon' \geq \epsilon - \frac{(q_{\mathcal{H}} + Nq_S + 1)^2}{2^{l_0}} - \frac{2q_S(q_{\mathcal{H}} + Nq_S) + q_{\mathcal{G}} + 1}{p}$$

and

$$t' \leq t + O(q_S t_{exp}).$$

The proof of this theorem is found in Appendix A.

7 Conclusion

At CCS'06, Bellare and Neven introduced the first multisignature scheme provably secure against rogue-key attacks in the plain public-key model. Their scheme is, however, loosely related to the CDH problem in the random-oracle model; the security proof was relied on the general forking lemma.

In this paper, we have presented two efficient multisignature schemes that are proven secure against rogue-key attack in the plain public-key model. Their security is also tightly related to either the CDH or the DDH problem in the random-oracle model. This means that they are almost as secure as Diffie-Hellman problems. Our signatures are the first to provide exact security while assuring security against rogue-key attacks in the plain public-key model.

Acknowledgments

The authors thank Gregory Neven for his detailed and helpful comments on the manuscript, and the anonymous referees for valuable feedback. This work was supported by Conseil Général des Landes and the French Ministry for Research under Project ANR-07-SESU-FLUOR.

References

1. Bagherzandi, A., Jarecki, S.: Multisignatures using proofs of secret key possession, as secure as the diffie-hellman problem. In: Ostrovsky, R., De Prisco, R., Visconti, I. (eds.) SCN 2008. LNCS, vol. 5229, pp. 218–235. Springer, Heidelberg (2008)

2. Barreto, P.S.L.M., Kim, H.Y., Lynn, B., Scott, M.: Efficient algorithms for pairing-based cryptosystems. In: Yung, M. (ed.) CRYPTO 2002. LNCS, vol. 2442, pp. 354–368. Springer, Heidelberg (2002)
3. Bellare, M., Namprempre, C., Neven, G.: Unrestricted aggregate signatures. In: Arge, L., Cachin, C., Jurdziński, T., Tarlecki, A. (eds.) ICALP 2007. LNCS, vol. 4596, pp. 411–422. Springer, Heidelberg (2007)
4. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: CCS 2006: Proceedings of the 13th ACM conference on Computer and communications security, pp. 390–399. ACM Press, New York (2006)
5. Boneh, D., Gentry, C., Lynn, B., Shacham, H.: Aggregate and verifiably encrypted signatures from bilinear maps. In: Biham, E. (ed.) EUROCRYPT 2003. LNCS, vol. 2656, pp. 416–432. Springer, Heidelberg (2003)
6. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. In: Boyd, C. (ed.) ASIACRYPT 2001. LNCS, vol. 2248, pp. 514–532. Springer, Heidelberg (2001)
7. Boyd, C.: Digital multisignatures. In: Cryptography and Coding, pp. 241–246. Oxford University Press, Oxford (1989)
8. Chaum, D., Pedersen, T.P.: Wallet Databases with Observers. In: Brickell, E.F. (ed.) CRYPTO 1992. LNCS, vol. 740, pp. 89–105. Springer, Heidelberg (1993)
9. Chevallier-Mames, B.: An Efficient CDH-Based Signature Scheme with a Tight Security Reduction. In: Shoup, V. (ed.) CRYPTO 2005. LNCS, vol. 3621, pp. 511–526. Springer, Heidelberg (2005)
10. Goh, E.-J., Jarecki, S.: A signature scheme as secure as the Diffie-Hellman problem. In: Biham, E. (ed.) EUROCRYPT 2003. LNCS, vol. 2656, pp. 401–415. Springer, Heidelberg (2003)
11. Goh, E.-J., Jarecki, S., Katz, J., Wang, N.: Efficient signature schemes with tight security reductions to the diffie-hellman problems. Journal of Cryptology 20(4), 493–514 (2007)
12. Itakura, K., Nakamura, K.: A public key cryptosystem suitable for digital multisignatures. NEC Research and Development 71, 1–8 (1983)
13. Jakobsson, M., Schnorr, C.-P.: Efficient Oblivious Proofs of Correct Exponentiation. In: CMS 1999: Communications and Multimedia Security. IFIP Conference Proceedings, vol. 152, pp. 71–86. Kluwer, Dordrecht (1999)
14. Joux, A., Nguyen, K.: Separating Decision Diffie-Hellman from Computational Diffie-Hellman in cryptographic groups. J. Cryptology 16(4), 239–247 (2003)
15. Katz, J., Wang, N.: Efficiency improvements for signature schemes with tight security reductions. In: CCS 2003: Proceedings of the 10th ACM Conference on Computer and Communications Security, pp. 155–164 (2003)
16. Micali, S., Ohta, K., Reyzin, L.: Accountable-subgroup multisignatures. In: CCS 2001: Proceedings of the 8th ACM conference on Computer and Communications Security, pp. 245–254. ACM Press, New York (2001)
17. Micali, S., Reyzin, L.: Improving the exact security of digital signature schemes. J. Cryptology 15(1), 1–18 (2002)
18. Okamoto, T.: A digital multisignature scheme using bijective public-key cryptosystems. ACM Trans. Comput. Syst. 6(4), 432–441 (1988)
19. Ristenpart, T., Yilek, S.: The power of proofs-of-possession: Securing multiparty signatures against rogue-key attacks. In: Naor, M. (ed.) EUROCRYPT 2007. LNCS, vol. 4515, pp. 228–245. Springer, Heidelberg (2007)

A Proof of Theorem 2

Proof. Assume we have a polynomial time forger that runs in time at most t, makes at most q_H hash queries and at most q_S signature queries and outputs a valid multisignature with probability at least ϵ. We need to construct an algorithm $\mathcal{B}$ which, by interacting with the forger $\mathcal{A}$, solves DDH problem with probability ϵ' within time t'.

Informally, the aim of Algorithm $\mathcal{B}$ is to determine whether a tuple (g, h, y_1, z_1) is a random tuple or a DH tuple. Assume that $\mathcal{A}$ is trying to attack the honest signer P^* who have the public keys $PK^* = (y_1, z_1)$. $\mathcal{B}$ sets $PK_1 = (y_1, z_1)$ and runs $\mathcal{A}$ on input PK^*. Algorithm $\mathcal{B}$ simulates the signing and hash oracle for $\mathcal{A}$ as follows:

First, algorithm $\mathcal{B}$ maintains initially empty associative lists H and G which are used to simulate random oracles $\mathcal{H}, \mathcal{G}$, respectively. We also make use of a list T which assigns a unique index $1 \leq i \leq q_H + Nq_S$ to each public key PK occurring either as a cosigner's public key in one of $\mathcal{A}$'s signature queries, or as the first item in the argument of one of $\mathcal{A}$'s queries to $\mathcal{G}$. Algorithm $\mathcal{B}$ uses a counter ctr indicating the current index of this list, initially set to zero. $\mathcal{B}$ assigns $\mathsf{T}[PK^*] \leftarrow 0$. It responds to $\mathcal{A}$'s oracle queries, essentially, at random as follows:

Queries to $\mathcal{H}$. In response to a query $\mathcal{H}(u_i)$ or $\mathcal{H}(v_i)$, $\mathcal{B}$ first checks if the output of $\mathcal{H}$ on this input has been previously defined. If so, $\mathcal{B}$ returns the previously assigned value. Otherwise, $\mathcal{B}$ returns with a value chosen uniformly at random from $\{0,1\}^{l_0}$. All queries u_i, v_i are stored in the list H.

Queries to $\mathcal{G}$. In response to a query $\mathcal{G}$, we first parse the argument of the query into two portions as PK and Q. If $\mathsf{T}[PK]$ is undefined then $\mathcal{B}$ increases ctr and sets $\mathsf{T}[PK] \leftarrow ctr$. If $\mathsf{G}[ctr, Q]$ is undefined, then $\mathcal{B}$ assigns $\mathsf{G}[i, Q]$, for all $1 \leq i \leq q_H + Nq_S$ with random numbers, and picks in advance at random $e_1, \ldots, e_{q_H+q_S} \in \mathbb{Z}_p$ to assign for $\mathsf{G}[0, Q]$.

Signing query on m with group of users L. Signature queries to the honest signer P^* consists of three rounds. First, the adversary provides m, L to P^* and receives the individual challenge h^*, t^* from P^* in response. Second, playing the role of rest signer, the adversary $\mathcal{A}$ provides the challenges h_i, t_i to P^* and receives u^*, v^* from P^* in response. Third, the adversary provides the commitments u_i, v_i to P^* and receives s^* from P^* in response. As stated above, in the simulation, it is not the adversary providing the joint commitment u, v to simulator, we do not thus need to use rewinding. In detail, answering signature queries works as follows:

First, $\mathcal{B}$ checks whether $P^* \notin L$, if so algorithm $\mathcal{B}$ returns $\perp$ to $\mathcal{A}$. If not, it parses the public keys of signers in L as $Pk = \{PK_1 = PK^*, PK_2, \ldots, PK_n\}$. Then, $\mathcal{B}$ checks whether $\mathsf{T}[PK_i]$, for $i \in \{2, \ldots, n\}$, has already been defined. If not, it increases ctr and sets $\mathsf{T}[PK_i] \leftarrow ctr$. Then, $\mathcal{B}$ sets c_1 at random

as $e_1, \ldots, e_{q_H+q_S}$ in advance. $\mathcal{B}$ generates $(\gamma, s_1) \in \mathbb{Z}_p^2$ at random, computes $u_1 = g^{s_1} y_1^{-c_1}$ and $v_1 = h^{s_1} z_1^{-c_1}$. It sets $h_1 = \mathcal{H}(u_1)$, $v_1 = \mathcal{H}(v_1)$ and sends to all signers.

After receiving $h_2, \cdots, h_n$ and $t_2, \cdots, t_n$ from the adversary $\mathcal{A}$, $\mathcal{B}$ looks up in the list H for values u_j, v_j such that $h_i = \mathcal{H}(u_j)$ and $t_i = \mathcal{H}(v_j)$. If multiple such values are found for some i, the algorithm $\mathcal{B}$ stops (**Event 1**). If no such value was found for some i then it sets $alert \leftarrow true$ and sends u_1, v_1 to all cosigners; otherwise, $\mathcal{B}$ computes $u = \prod_{i=1}^n u_i$ and $v = \prod_{i=1}^n v_i$. Then, $\mathcal{B}$ checks whether $\mathsf{G}[0, Q]$ has already been defined for $Q = \langle u, v, Pk, m, g, h\rangle$. If so, it fails and stops (**Event 2**). If not, it sets $\mathcal{G}(PK_1, u, v, Pk, m, g, h) = \mathsf{G}[0, Q] = c_1$, randomly chooses $\mathsf{G}[i, Q] \overset{R}{\leftarrow} \mathbb{Z}_p$ for all $1 \leq i \leq q_H + Nq_S$ and sends u_1, v_1 to all cosigners.

After receiving $u_2, v_2, \ldots, u_n, v_n$ from $\mathcal{A}$, $\mathcal{B}$ verifies that $h_i = \mathcal{H}(u_i)$ and $t_i = \mathcal{H}(v_i)$ for all $1 \leq i \leq n$. If not, it returns $\perp$ to $\mathcal{A}$. If $alert = true$, $\mathcal{B}$ fails and stops (**Event 3**). Else, it sends s_1 to all cosigners.

After receiving $s_2, \cdots, s_n$ from cosigners ($\mathcal{A}$), $\mathcal{B}$ computes $s = \sum_{i=1}^n s_i$ and returns the valid signature (u, v, s).

As we can see, this simulator is valid, except for some events:

- **Event 1:** In this case, there exist two values $u_i \neq u_i'$ or $v_i \neq v_i'$ such that $h_i = \mathcal{H}(u_i) = \mathcal{H}(u_i')$ or $t_i = \mathcal{H}(v_i) = \mathcal{H}(v_i')$ for some i, i.e, there is at least one collision occurred in $\mathcal{H}$. As outputs of $\mathcal{H}$ are chosen at random from $\{0,1\}^{l_0}$ and since there are at most $q_\mathcal{H} + Nq_S$ queries to $\mathcal{H}$, the probability that at least one collision occurs is upper bounded by $\frac{(q_\mathcal{H}+Nq_S)(q_\mathcal{H}+Nq_S+1)/2}{2^{l_0}} \leq \frac{(q_\mathcal{H}+Nq_S+1)^2}{2^{l_0+1}}$.
- **Event 2:** Algorithm $\mathcal{B}$ only aborts at event 2 if it has run into an input string $\langle 0, u, v, Pk, m, g, h\rangle$ on which $\mathcal{G}$ has been already queried. We distinguish between the case that $\mathcal{H}(u_1)$ and $\mathcal{H}(v_1)$ were previously queried by the forger, and the case that they were not. In the first case, $\mathcal{A}$ probably knows u, v and may have deliberately queried $\mathcal{G}(PK, u, v, Pk, m, g, h)$ for some PK. But since u_1, v_1 was chosen by $\mathcal{B}$ independently from $\mathcal{A}$'s view at the beginning of the signing protocol, the probability that $\mathcal{A}$ queried $\mathcal{H}(u_1)$ and $\mathcal{H}(v_1)$ is at most $\frac{q_\mathcal{H}+Nq_S}{p}$, for one signature query. In the second case, $\mathcal{A}$'s view is completely independent of u_1 and v_1, and hence of u and v. The probability that u and v occurred by chance in a previous query to $\mathcal{G}$ or was set by $\mathcal{B}$ in one of the $i-1$ previous signature simulations is at most $\frac{q_\mathcal{G}+q_S}{p}$, for one signature query. For q_S signature queries, the failure probability is thus upper bounded by $\frac{q_S((q_\mathcal{H}+Nq_S)+(q_\mathcal{G}+q_S))}{p} \leq \frac{q_S(q_H+(N+1)q_S)}{p}$, where $q_H = q_\mathcal{H} + q_\mathcal{G}$.
- **Event 3:** In this case, $\mathcal{A}$ must have predicted the value of $\mathcal{H}(u_i)$ or $\mathcal{H}(v_i)$ for at least one $1 \leq i \leq n$, which it can do with probability at most $\frac{N}{2^{l_0}}$, for one signature query. For q_S signature queries, the failure probability is thus upper bounded by $\frac{q_S N}{2^{l_0}}$.

As a conclusion, except with a failure probability:

$$\begin{aligned}\delta_{stop} &= \frac{(q_{\mathcal{H}} + Nq_S + 1)^2}{2^{l_0+1}} + \frac{q_S(q_H + (N+1)q_S)}{p} + \frac{q_S N}{2^{l_0}} \\ &\leq \frac{(q_{\mathcal{H}} + Nq_S + 1)^2}{2^{l_0}} + \frac{q_S(q_H + (N+1)q_S)}{p},\end{aligned}$$

the simulation is successful.

Eventually, $\mathcal{A}$ halts and outputs an attempted forgery $\sigma = (\hat{u}, \hat{v}, \hat{s}, \{s_i\}_{i=2..n}))$ on some message $\hat{m}$ along with $L = \{P^*, P_2, \cdots, P_n\}$. It must not previously have requested a signature on $\hat{m}$ with L. In addition, it outputs the private keys $(x_2, \cdots, x_n)$ for all secret keys except the key x of the challenge P^*. Algorithm $\mathcal{B}$ first computes additional random oracle queries $\mathcal{G}_1(PK_i, \hat{u}, \hat{v}, Pk, \hat{m}, g, h)$ for $1 \leq i \leq n$, thereby making sure that $\mathsf{T}[PK_i]$ is defined.

If $\mathcal{A}$'s forgery is valid, i.e. g, h, y, z, where $y = \prod_{i=1}^n y_i$ and $z = \prod_{i=1}^n z_i$ and (g, h, y_i, z_i) for each P_i are Diffie-Hellman tuples, and then (g, h, y_1, z_1) is a Diffie-Hellman tuple, the algorithm $\mathcal{B}$ outputs 1 with the probability $\epsilon - \delta_{stop}$; otherwise it outputs 0.

On the other hand, if (g, h, y, z) is a random tuple, then it is not a Diffie-Hellman tuple with probability $1 - 1/p$. In this case, for any u, v and any query $\mathcal{G}(PK_1, u, v, Pk, m, g, h)$ made by $\mathcal{A}$ then there is at most one possible value of c for which there exists an s satisfying $u = g^s y_1^c$ and $v = h^s y_2^c$ (*lemma* 1 in [11]). Thus, $\mathcal{A}$ outputs a forgery (and hence $\mathcal{B}$ outputs 1) with probability at most $\frac{1}{p} + \frac{q_{\mathcal{G}}}{p} \leq \frac{q_{\mathcal{G}}+1}{p}$.

Summing the probabilities, we see that:

$$\begin{aligned}&|Pr\left[\mathcal{B}(g, g^x, g^y, g^{xy}) = 1\right] - Pr\left[\mathcal{B}(g, g^x, g^y, g^z) = 1\right] : x, y, z \leftarrow \mathbb{Z}_p| \\ &\geq \epsilon - \delta_{stop} - \frac{q_{\mathcal{G}} + 1}{p} \\ &\geq \epsilon - \frac{(q_{\mathcal{H}} + Nq_S + 1)^2}{2^{l_0}} - \frac{q_S(q_H + (N+1)q_S)}{p} - \frac{(q_{\mathcal{G}} + 1)}{p} \\ &\geq \epsilon - \frac{(q_{\mathcal{H}} + Nq_S + 1)^2}{2^{l_0}} - \frac{q_S(q_H + (N+1)q_S) + q_{\mathcal{G}} + 1}{p}\end{aligned}$$

and the running time t' satisfies

$$t' \leq t + O(q_S t_{exp}),$$

where t_{exp} is the time of an exponentiation in $\mathbb{G}$.